

CODIGO MEMORIA (FIN 1 GPT-5.5)

Perfecto, te voy a generar el proyecto completo con todo el código y los archivos de configuración de memoria.

Voy a estructurarlo en bloques que puedas copiar directamente:

BLOQUE 1: Archivos Base del Proyecto

requirements.txt

txt

```
# Core
chromadb>=0.4.22
sentence-transformers>=2.2.2
pydantic>=2.0
python-dotenv>=1.0.0

# Testing
pytest>=7.0.0
pytest-asyncio>=0.21.0

# Optional (para UI)
gradio>=4.0.0
```

memory_config.py

Python

```
from pathlib import Path
from pydantic import BaseModel
from typing import List

class MemoryConfig(BaseModel):
    """Configuración centralizada del sistema de memoria"""

    # Rutas
    persist_directory: str = "./memory/chroma"
    episodic_path: str = "./memory/episodic"
    semantic_path: str = "./memory/semantic"
    backup_path: str = "./.ai_memory"

    # Modelos
```

```

embedding_model: str = "all-MiniLM-L6-v2"

# Límites
max_stm_items: int = 20
consolidation_threshold: int = 10
retrieval_limit: int = 10

# Auto-consolidation
auto_consolidate: bool = True
consolidate_after_n: int = 5

# Confianza mínima
min_confidence: float = 0.7

config = MemoryConfig()

```

BLOQUE 2: Núcleo del Sistema de Memoria

```
ai_architect/__init__.py
```

Python

```

from .core.memory import MemorySystem
from .config.settings import settings

__version__ = "1.0.0"
__all__ = ["MemorySystem", "settings"]

```

```
ai_architect/core/__init__.py
```

Python

```

from .memory import MemorySystem
from .mcp_server import create_server

__all__ = ["MemorySystem", "create_server"]

```

```
ai_architect/core/memory.py
```

Python

```
"""
```

Sistema de Memoria Híbrida

=====

Combina memoria de corto plazo (STM), largo plazo (LTM) y episódica.

Usa ChromaDB para búsqueda vectorial eficiente.

"""

```
import chromadb
from chromadb.utils import embedding_functions
from datetime import datetime
import json
import os
from typing import List, Dict, Optional
from dataclasses import dataclass, asdict
from pathlib import Path
```

@dataclass

```
class Episode:
    """Representa una memoria episódica"""
    timestamp: str
    project: str
    decision: str
    context: str
    outcome: str
    tags: List[str]
    lessons_learned: List[str] = None

    def to_dict(self) -> dict:
        return asdict(self)
```

@dataclass

```
class Lesson:
    """Representa una lección aprendida"""
    id: str
    content: str
    category: str
    confidence: float
    timestamp: str
    source: str = "automatic"
    validated: bool = False

    def to_dict(self) -> dict:
        return asdict(self)
```

class MemorySystem:

```
    """
    Sistema de memoria con tres niveles:
    - STM: Memoria de corto plazo (sesión actual)
    - LTM: Memoria de largo plazo (ChromaDB)
    - Episódica: Historial de decisiones y resultados
    """
```

```
    def __init__(self, persist_directory: str = "./memory/chroma"):
        # === ChromaDB Setup ===
        self.client = chromadb.PersistentClient(path=persist_directory)
        self.embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
            model_name="all-MiniLM-L6-v2"
        )

        # Colecciones
        self.episodes_collection = self.client.get_or_create_collection(
            name="episodes",
            embedding_function=self.embedding_fn
        )
```

```

self.lessons_collection = self.client.get_or_create_collection(
    name="lessons",
    embedding_function=self.embedding_fn
)

self.patterns_collection = self.client.get_or_create_collection(
    name="patterns",
    embedding_function=self.embedding_fn
)

# STM (en memoria)
self.stm: List[Dict] = []
self.stm_max_size = 20

# Rutas
self.episodic_path = Path("./memory/episodic")
self.semantic_path = Path("./memory/semantic")
self.episodic_path.mkdir(parents=True, exist_ok=True)
self.semantic_path.mkdir(parents=True, exist_ok=True)

# Contadores
self.session_interactions = 0

# ===== STM (Short Term Memory) =====

def add_to_stm(self, item: Dict) -> None:
    """Añade un elemento a la memoria de corto plazo"""
    item['stm_timestamp'] = datetime.now().isoformat()
    self.stm.append(item)
    self.session_interactions += 1

# Limitar tamaño de STM
if len(self.stm) > self.stm_max_size:
    self._consolidate_stm_to_ltm()

def get_stm(self, n: int = None) -> List[Dict]:
    """Obtiene los últimos n elementos de STM"""
    if n is None:
        return self.stm.copy()
    return self.stm[-n:]

def clear_stm(self) -> None:
    """Limpia la memoria de corto plazo"""
    self.stm = []

def _consolidate_stm_to_ltm(self) -> None:
    """Consolida STM a LTM cuando se llena"""
    if len(self.stm) >= self.stm_max_size:
        # Mover los más antiguos a LTM como lecciones
        old_items = self.stm[:-10] # Mantener últimos 10
        for item in old_items:
            if 'learned' in item:
                self.store_lesson(
                    lesson=item['learned'],
                    category=item.get('category', 'general'),
                    source="stm_consolidation"
                )
        self.stm = self.stm[-10:]

# ===== Episodic Memory =====

def store_episode(

```

```

        self,
        project: str,
        decision: str,
        context: str,
        outcome: str,
        tags: List[str] = None,
        lessons_learned: List[str] = None
    ) -> str:
        """Guarda una memoria episódica con todos sus detalles"""

        episode = Episode(
            timestamp=datetime.now().isoformat(),
            project=project,
            decision=decision,
            context=context,
            outcome=outcome,
            tags=tags or [],
            lessons_learned=lessons_learned or []
        )

        # Guardar en archivo JSON
        filename = f"{datetime.now().strftime('%Y%m%d_%H%M%S')}_{project.replace(' ',
        '_')}.json"
        filepath = self.episodic_path / filename

        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(episode.to_dict(), f, indent=2, ensure_ascii=False)

        # Indexar en ChromaDB
        search_text = f"""
        Proyecto: {project}
        Decisión: {decision}
        Contexto: {context}
        Resultado: {outcome}
        Lecciones: {' '.join(lessons_learned or [])}
        """

        self.episodes_collection.add(
            documents=[search_text],
            metadatas=[{
                "type": "episode",
                "project": project,
                "tags": json.dumps(tags),
                "timestamp": episode.timestamp,
                "filename": filename
            }],
            ids=[filename]
        )

        # Auto-guardar lecciones aprendidas
        if lessons_learned:
            for lesson in lessons_learned:
                self.store_lesson(lesson, category="episode_derived", source=filename)

        return filename

def get_episodes(self, project: str = None, limit: int = 10) -> List[Episode]:
    """Obtiene episodios, opcionalmente filtrados por proyecto"""
    if project:
        results = self.episodes_collection.get(
            where={"project": project},
            include=["documents", "metadatas"]
        )

```

```

else:
    results = self.episodes_collection.get(
        include=["documents", "metadatas"]
    )

    episodes = []
    for i, metadata in enumerate(results['metadatas'][:limit]):
        filepath = self.episodic_path / metadata['filename']
        if filepath.exists():
            with open(filepath, 'r', encoding='utf-8') as f:
                data = json.load(f)
                episodes.append(Episode(**data))
    return episodes

# ===== Long Term Memory (LTM) =====

def store_lesson(
    self,
    lesson: str,
    category: str = "general",
    confidence: float = 0.7,
    source: str = "unknown"
) -> str:
    """Guarda una lección aprendida en LTM"""

    lesson_id = f"lesson_{datetime.now().strftime('%Y%m%d%H%M%S')}_{hash(lesson) %
10000}"

    self.lessons_collection.add(
        documents=[lesson],
        metadatas=[{
            "type": "lesson",
            "category": category,
            "confidence": confidence,
            "timestamp": datetime.now().isoformat(),
            "source": source,
            "validated": "false"
        }],
        ids=[lesson_id]
    )

    return lesson_id

def store_pattern(
    self,
    pattern: str,
    context: str,
    effectiveness: float
) -> str:
    """Guarda un patrón arquitectónico o de diseño"""

    pattern_id = f"pattern_{datetime.now().strftime('%Y%m%d%H%M%S')}"

    self.patterns_collection.add(
        documents=[f"{pattern}\n\nContexto: {context}"],
        metadatas=[{
            "type": "pattern",
            "effectiveness": effectiveness,
            "timestamp": datetime.now().isoformat()
        }],
        ids=[pattern_id]
    )

```

```

        return pattern_id

# ===== Retrieval =====

def retrieve(
    self,
    query: str,
    collection: str = "all",
    n_results: int = 5,
    min_confidence: float = 0.0
) -> List[Dict]:
    """
    Recupera memorias relevantes usando búsqueda vectorial.

    Args:
        query: Consulta de búsqueda
        collection: 'episodes', 'lessons', 'patterns', o 'all'
        n_results: Número máximo de resultados
        min_confidence: Filtrar por confianza mínima

    Returns:
        Lista de memorias relevantes con scores
    """
    results = []

    collections_to_search = {
        "episodes": self.episodes_collection,
        "lessons": self.lessons_collection,
        "patterns": self.patterns_collection
    }

    if collection == "all":
        search_collections = collections_to_search
    else:
        search_collections = {collection: collections_to_search.get(collection)}

    for name, col in search_collections.items():
        if col is None:
            continue

        query_results = col.query(
            query_texts=[query],
            n_results=n_results,
            include=["documents", "metadatas", "distances"]
        )

        for i in range(len(query_results['documents'][0])):
            metadata = query_results['metadatas'][0][i]

            # Filtrar por confianza si aplica
            if 'confidence' in metadata:
                if metadata['confidence'] < min_confidence:
                    continue

            # Calcular score de relevancia (invertimos distancia)
            distance = query_results['distances'][0][i]
            relevance = 1 - (distance / 2) # Normalizar a 0-1

            results.append({
                "content": query_results['documents'][0][i],
                "metadata": metadata,
                "collection": name,
                "relevance": round(relevance, 3),
            })

```

```

        "id": query_results['ids'][0][i]
    })

    # Ordenar por relevancia
    results.sort(key=lambda x: x['relevance'], reverse=True)

    return results[:n_results]

# ===== Consolidation =====

def consolidate(self) -> Dict:
    """
    Ejecuta consolidación de memorias.
    Une memorias similares y refuerza las validadas.
    """
    stats = {
        "episodes_processed": 0,
        "lessons_merged": 0,
        "patterns_updated": 0
    }

    # Obtener todas las lecciones
    all_lessons = self.lessons_collection.get(
        include=["documents", "metadatas"]
    )

    # Buscar lecciones similares para consolidar
    seen_ids = set()
    for i, (doc, metadata, id) in enumerate(zip(
        all_lessons['documents'],
        all_lessons['metadatas'],
        all_lessons['ids']
    )):
        if id in seen_ids:
            continue

        # Buscar lecciones similares
        similar = self.lessons_collection.query(
            query_texts=[doc],
            n_results=3,
            include=["metadatas"]
        )

        for j, sim_metadata in enumerate(similar['metadatas'][0]):
            if j == 0: # Skip self
                continue

            # Si tiene alta similitud, marcar para revisión
            if similar['distances'][0][j] < 0.2: # Muy similar
                seen_ids.add(similar['ids'][0][j])
                stats["lessons_merged"] += 1

    stats["episodes_processed"] = len(all_lessons['ids'])

    return stats

def get_statistics(self) -> Dict:
    """Obtiene estadísticas del sistema de memoria"""
    return {
        "episodes_count": len(self.episodes_collection.get()['ids']),
        "lessons_count": len(self.lessons_collection.get()['ids']),
        "patterns_count": len(self.patterns_collection.get()['ids']),
        "stm_size": len(self.stm),
    }

```



```
        "session_interactions": self.session_interactions
    }
```

BLOQUE 3: Servidor MCP para OpenCode

ai_architect/core/mcp_server.py

Python

```
"""
Servidor MCP para integrar con OpenCode/Cursor
=====
Expone las herramientas del sistema de memoria como herramientas MCP.
"""

from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent
from pydantic import AnyUrl
import json

from .memory import MemorySystem

# Inicializar servidor
server = Server("ai-memory-system")

# Crear instancia global del sistema de memoria
memory_system = MemorySystem()

@server.list_tools()
async def list_tools() -> list[Tool]:
    """Lista todas las herramientas disponibles"""
    return [
        Tool(
            name="store_episode",
            description="Guarda una decisión importante del proyecto como memoria episódica",
            inputSchema={
                "type": "object",
                "properties": {
                    "project": {"type": "string", "description": "Nombre del proyecto"},
                    "decision": {"type": "string", "description": "La decisión tomada"},
                    "context": {"type": "string", "description": "Contexto de la decisión"},
                    "outcome": {"type": "string", "description": "Resultado obtenido"},
                    "tags": {"type": "array", "items": {"type": "string"}, "description": "Tags para categorizar"},
                    "lessons_learned": {"type": "array", "items": {"type": "string"}, "description": "Lecciones derivadas"}
                },
                "required": ["project", "decision", "context", "outcome"]
            }
        )
    ]
```

```

    }
  ),
  Tool(
    name="store_lesson",
    description="Guarda una lección aprendida en la memoria de largo plazo",
    inputSchema={
      "type": "object",
      "properties": {
        "lesson": {"type": "string", "description": "La lección aprendida"},
        "category": {"type": "string", "description": "Categoría: general,
arquitectura, código, prompt"},
        "confidence": {"type": "number", "description": "Nivel de confianza
0-1"}
      },
      "required": ["lesson"]
    }
  ),
  Tool(
    name="store_pattern",
    description="Guarda un patrón arquitectónico o de diseño",
    inputSchema={
      "type": "object",
      "properties": {
        "pattern": {"type": "string", "description": "Descripción del patrón"},
        "context": {"type": "string", "description": "Cuándo aplicar este
patrón"},
        "effectiveness": {"type": "number", "description": "Efectividad 0-1"}
      },
      "required": ["pattern", "context"]
    }
  ),
  Tool(
    name="retrieve_memory",
    description="Busca en la memoria del sistema para encontrar información
relevante",
    inputSchema={
      "type": "object",
      "properties": {
        "query": {"type": "string", "description": "Consulta de búsqueda"},
        "collection": {"type": "string", "enum": ["all", "episodes", "lessons",
"patterns"], "description": "Dónde buscar"},
        "n_results": {"type": "number", "description": "Número de resultados"}
      },
      "required": ["query"]
    }
  ),
  Tool(
    name="get_stm",
    description="Obtiene la memoria de corto plazo de la sesión actual",
    inputSchema={
      "type": "object",
      "properties": {
        "n": {"type": "number", "description": "Número de últimos elementos"}
      }
    }
  ),
  Tool(
    name="add_to_stm",
    description="Añade un elemento a la memoria de corto plazo",
    inputSchema={
      "type": "object",
      "properties": {
        "content": {"type": "string", "description": "Contenido a guardar"},

```

```

        "learned": {"type": "string", "description": "Qué aprendió de esto"},
        "category": {"type": "string", "description": "Categoría"}
    },
    "required": ["content"]
}
),
Tool(
    name="get_memory_stats",
    description="Obtiene estadísticas del sistema de memoria"
),
Tool(
    name="consolidate_memory",
    description="Ejecuta consolidación de memorias"
)
]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    """Ejecuta una herramienta del sistema de memoria"""

    try:
        if name == "store_episode":
            result = memory_system.store_episode(**arguments)
            return [TextContent(type="text", text=json.dumps({
                "status": "success",
                "episode_id": result
            }, indent=2))]

        elif name == "store_lesson":
            result = memory_system.store_lesson(**arguments)
            return [TextContent(type="text", text=json.dumps({
                "status": "success",
                "lesson_id": result
            }, indent=2))]

        elif name == "store_pattern":
            result = memory_system.store_pattern(**arguments)
            return [TextContent(type="text", text=json.dumps({
                "status": "success",
                "pattern_id": result
            }, indent=2))]

        elif name == "retrieve_memory":
            results = memory_system.retrieve(**arguments)
            return [TextContent(type="text", text=json.dumps({
                "status": "success",
                "results": results,
                "count": len(results)
            }, indent=2))]

        elif name == "get_stm":
            stm = memory_system.get_stm(arguments.get('n'))
            return [TextContent(type="text", text=json.dumps({
                "status": "success",
                "stm": stm,
                "count": len(stm)
            }, indent=2))]

        elif name == "add_to_stm":
            memory_system.add_to_stm(arguments)
            return [TextContent(type="text", text=json.dumps({
                "status": "success",
                "message": "Añadido a STM"
            }, indent=2))]

```

```

        }, indent=2))

    elif name == "get_memory_stats":
        stats = memory_system.get_statistics()
        return [TextContent(type="text", text=json.dumps({
            "status": "success",
            "stats": stats
        }, indent=2))]

    elif name == "consolidate_memory":
        stats = memory_system.consolidate()
        return [TextContent(type="text", text=json.dumps({
            "status": "success",
            "consolidation_stats": stats
        }, indent=2))]

    else:
        return [TextContent(type="text", text=json.dumps({
            "status": "error",
            "message": f"Herramienta desconocida: {name}"
        }, indent=2))]

except Exception as e:
    return [TextContent(type="text", text=json.dumps({
        "status": "error",
        "error": str(e)
    }, indent=2))]

def create_server():
    """Crea y retorna el servidor MCP"""
    return server

async def main():
    """Punto de entrada para ejecutar el servidor"""
    async with stdio_server() as (read_stream, write_stream):
        await server.run(
            read_stream,
            write_stream,
            server.create_initialization_options()
        )

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())

```

ai_architect/config/__init__.py

Python

```

from .settings import settings

__all__ = ["settings"]

```

ai_architect/config/settings.py

Python

```
import os
from pathlib import Path
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    """Configuración centralizada del proyecto"""

    # Rutas
    project_root: Path = Path("./")
    memory_dir: Path = Path("./memory")
    chroma_dir: Path = Path("./memory/chroma")
    episodic_dir: Path = Path("./memory/episodic")
    semantic_dir: Path = Path("./memory/semantic")
    backup_dir: Path = Path("./.ai_memory")

    # Modelos
    embedding_model: str = "all-MiniLM-L6-v2"

    # Límites
    max_stm_items: int = 20
    consolidation_threshold: int = 10
    retrieval_limit: int = 10
    min_confidence: float = 0.7

    # Auto-consolidation
    auto_consolidate: bool = True
    consolidate_after_n: int = 5

    # API Keys (se cargan desde .env)
    groq_api_key: str = ""
    openrouter_api_key: str = ""
    google_api_key: str = ""

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"

settings = Settings()
```

BLOQUE 4: Archivos de Configuración de Memoria

```
.ai/memory/README.md
```

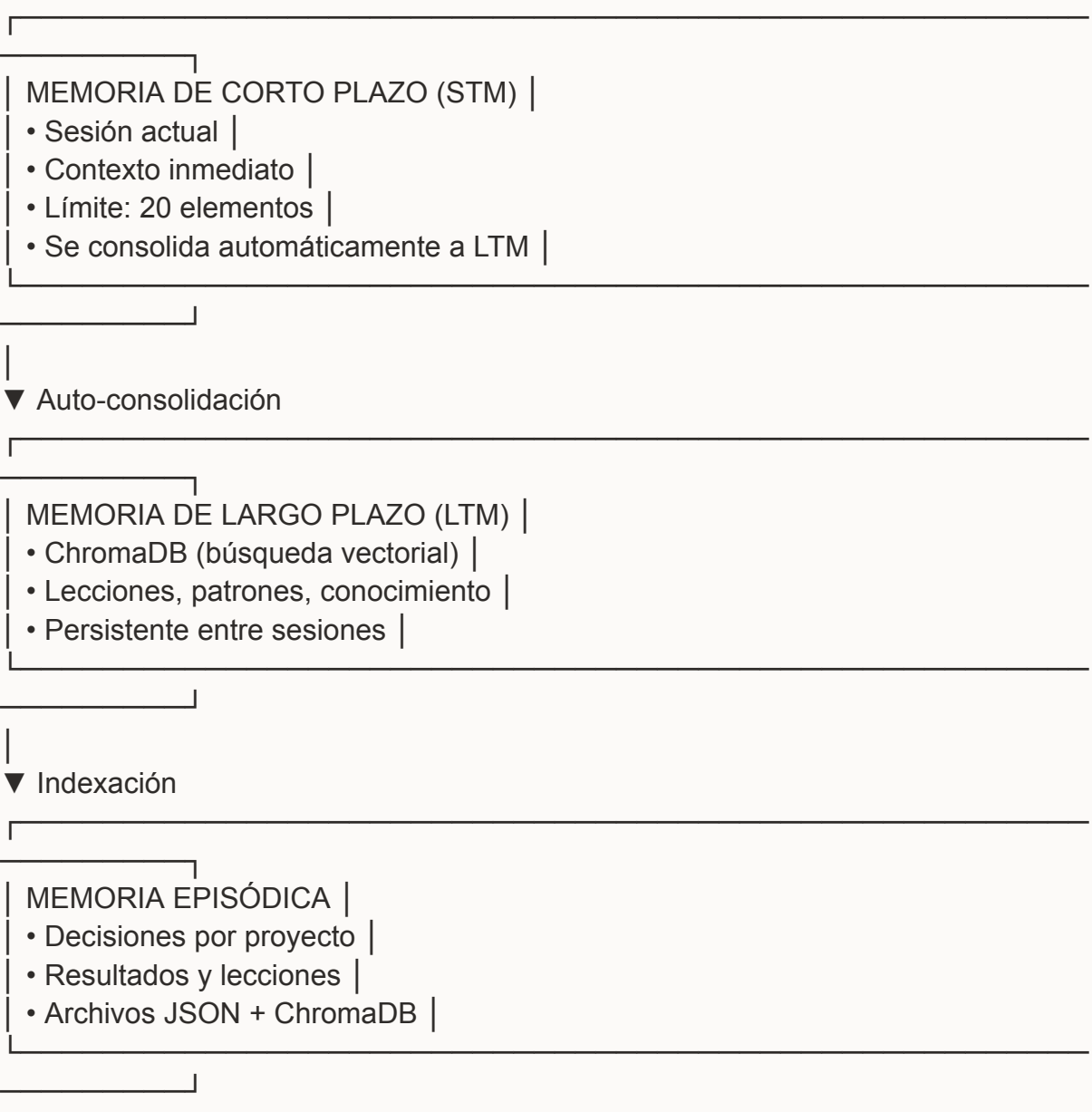
Markdown

```
# Sistema de Memoria AI
```

Visión General

Este sistema de memoria permite que el modelo de IA acumule conocimiento y experiencia a lo largo del tiempo, mejorando progresivamente su desempeño sin depender de un modelo específico.

Arquitectura de Tres Niveles



text

Archivos de Configuración

Archivo	Propósito
`STM.md`	Instrucciones para memoria de corto plazo
`LTM.md`	Instrucciones para memoria de largo plazo

```

| `episodic.md` | Sistema de memoria episódica |
| `consolidation.md` | Reglas de consolidación |
| `retrieval.md` | Estrategias de recuperación |
| `index.md` | Índice maestro del sistema |

## Uso Rápido

```python
from ai_architect import MemorySystem

mem = MemorySystem()

Guardar una decisión
mem.store_episode(
 project="mi-proyecto",
 decision="Usar arquitectura modular",
 context="Proyecto grande con múltiples componentes",
 outcome="Mejor mantenibilidad y escalabilidad",
 lessons_learned=["Separar concerns temprano"]
)

Buscar conocimiento relevante

results = mem.retrieve("arquitectura modular proyectos grandes")

```

# Integración con OpenCode

## 1. Configurar MCP en `opencode.json`:

JSON

```

{
 "mcpServers": {
 "ai-memory": {
 "command": "python",
 "args": ["ai_architect/core/mcp_server.py"]
 }
 }
}

```

## 2. Usar herramientas disponibles:

- `store_episode`: Guardar decisiones
- `store_lesson`: Guardar lecciones
- `retrieve_memory`: Buscar en memoria
- `add_to_stm`: Añadir a corto plazo

text

```
`.ai/memory/STM.md`

```markdown
# Memoria de Corto Plazo (STM)

## Propósito

Mantener contexto relevante de la sesión actual sin sobrecargar el contexto del modelo.

## Reglas de Uso

### Cuándo Usar STM

1. Contexto inmediato: Información relevante solo para la sesión actual
  - Archivos que se están editando actualmente
  - Decisiones parciales que aún no están consolidadas
  - Resultados de herramientas en ejecución

2. Seguimiento de tareas: Mantener estado de operaciones multi-paso
  - Pasos completados en una refactorización
  - Archivos pendientes por procesar
  - Decisiones intermedias

3. Tmp-analizaciones: Información que será relevante brevemente
  - Resultados de búsquedas
  - Contenido de archivos abiertos
  - Errores siendo debuggeados

### Cuándo NO Usar STM

1. ❌ Conocimiento generalizable a otros proyectos
2. ❌ Decisiones arquitectónicas importantes
3. ❌ Lecciones aprendidas permanentes
4. ❌ Patrones que podrían reutilizarse

## Límites

- Máximo de elementos: 20 items
- Auto-consolidación: Cuando se alcanza el límite, los 10 más antiguos se mueven a LTM
- TTL implícito: Se asume que después de 50 interacciones sin referencia, el item perdió relevancia

## Proceso de Consolidación
```

Cuando STM alcanza 20 elementos:

1. Identificar los 10 items más antiguos
2. Para cada item:
 - a. Si contiene "learned" → crear lección en LTM
 - b. Si es decisión importante → crear episodio
 - c. Si es patrón reutilizable → guardar en patterns

3. Mantener solo los 10 items más recientes
4. Registrar consolidación en logs

text

Ejemplo de Contenido STM

```
```json
{
 "type": "context",
 "content": "Editando auth/decorators.py",
 "category": "current_file",
 "timestamp": "2024-01-15T10:30:00Z"
}

{
 "type": "decision_partial",
 "content": "Decidí usar decoradores para autenticación",
 "category": "decision",
 "status": "pending_review",
 "timestamp": "2024-01-15T10:35:00Z"
}

{
 "type": "tool_result",
 "tool": "grep",
 "result": "Encontrados 5 archivos usando auth obsoleto",
 "category": "search_result",
 "timestamp": "2024-01-15T10:40:00Z"
}

}
```

## Integración con LTM

STM debe revisar LTM cuando:

1. Comienza una nueva sesión
2. Se detecta un proyecto conocido
3. Se necesitan contexto histórico

Ver: `LTM.md` para reglas de integración.

text

```
`.ai/memory/LTM.md`

```markdown
# Memoria de Largo Plazo (LTM)

## Propósito

Almacenar conocimiento persistente que mejora el desempeño del modelo a lo largo del tiempo, independientemente del proyecto o sesión.

## Componentes

### 1. Lecciones (Lessons)

**Qué guardar:**
- Insights sobre arquitecturas exitosas
- Errores comunes y cómo evitarlos
- Mejores prácticas validadas
- Patrones de diseño probados

**Estructura:**
```json
{
 "id": "lesson_20240115_143022_4521",
 "content": "Separar lógica de presentación de negocio mejora mantenibilidad",
 "category": "arquitectura",
 "confidence": 0.85,
 "timestamp": "2024-01-15T14:30:22Z",
 "source": "episodic_20240115_mi_proyecto",
 "validated": true
}
```

## Niveles de Confianza:

- 0.9-1.0: Validado múltiples veces
- 0.7-0.9: Validado al menos una vez
- 0.5-0.7: Hipótesis sin validar
- <0.5: No recomendado sin más evidencia

## 2. Patrones (Patterns)

### Qué guardar:

- Soluciones arquitectónicas recurrentes
- Estructuras de proyecto exitosas
- Configuraciones optimizadas
- Combinaciones de herramientas probadas

### Estructura:

JSON

```
{
 "id": "pattern_20240115_150000",
 "content": "Microservicios con API Gateway centralizado",
 "context": "Proyectos con múltiples servicios independientes",
 "effectiveness": 0.92,
 "applications": ["proyecto-a", "proyecto-b"],
 "timestamp": "2024-01-15T15:00:00Z"
}
```

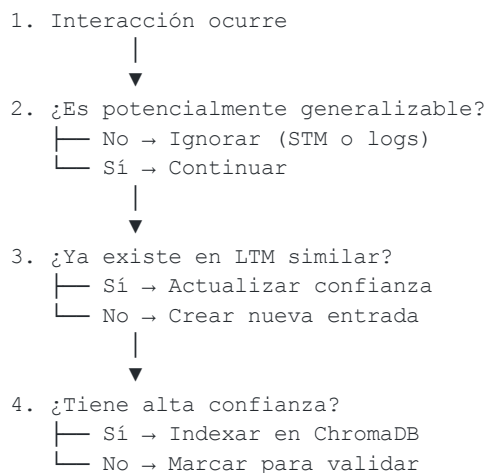
### 3. Conocimiento Técnico

Categorías:

- **arquitectura:** Patrones y estructuras
- **codigo:** Buenas prácticas de implementación
- **prompt:** Estrategias de prompting
- **debugging:** Técnicas de solución de problemas
- **workflow:** Procesos y metodologías

## Proceso de Adquisición

text



# Actualización de Confianza

Aumentar confianza:

- +0.1 por cada uso exitoso del patrón/lección
- +0.2 si se valida en múltiples proyectos
- +0.3 si expertos lo confirman

Reducir confianza:

- -0.1 por cada falla al aplicar
- -0.2 si surge alternativa mejor
- -0.3 si causa problemas inesperados

# Consulta de LTM

Antes de cada decisión importante:

1. Buscar lecciones relevantes al contexto
2. Buscar patrones aplicados exitosamente
3. Considerar confianza de cada resultado
4. Sugerir basándose en mayor confianza

Ver: `retrieval.md` para estrategias de búsqueda.

text

```
`.ai/memory/retrieval.md`
```

```
```markdown
```

```
# Estrategias de Recuperación de Memoria
```

```
## Principios Fundamentales
```

1. **Relevancia > Recencia**: No siempre lo más reciente es lo más útil
2. **Contexto es clave**: La misma query puede necesitar diferentes estrategias
3. **Diversidad**: Buscar en múltiples colecciones mejora resultados
4. **Confidence filtering**: Filtrar por debajo del umbral mínimo

Estrategias por Tipo de Consulta

1. Consultas de Decisión

****Cuando necesito decidir algo:****

1. Buscar en "episodes": decisiones similares y sus resultados
2. Buscar en "patterns": soluciones aplicadas antes
3. Buscar en "lessons": principios relevantes
4. Priorizar: decisiones > patrones > lecciones

text

****Query ejemplo:**** "arquitectura para proyecto con múltiples APIs"

2. Consultas de Error

****Cuando enfrento un error:****

1. Buscar en "lessons": errores similares y soluciones
2. Buscar en "episodes": cómo se resolvieron problemas antes
3. Filtrar por categoría: debugging, error-fixing

text

****Query ejemplo:**** "error timeout API externa solución"

3. Consultas de Mejores Prácticas

****Cuando busco optimizar:****

1. Buscar en "patterns": soluciones probadas
2. Buscar en "lessons": insights sobre eficiencia
3. Filtrar por confidence > 0.8

text

****Query ejemplo:**** "mejores prácticas API REST"

4. Consultas de Contexto de Proyecto

****Cuando retomo un proyecto:****

1. Buscar episodios del proyecto específico
2. Buscar patrones usados en proyectos similares
3. Revisar lecciones con source = proyecto

text

****Query ejemplo:**** "decisiones proyecto-mi-proyecto"

Parámetros de Búsqueda

Collection

Valor	Cuándo usarlo
`all`	Búsqueda general, sin contexto específico
`episodes`	Decisiones y resultados históricos
`lessons`	Conocimiento y principios
`patterns`	Soluciones arquitectónicas

Número de Resultados

Contexto	n_results
Decisión importante	10
Búsqueda general	5
Confirmación rápida	3
Exploración	15

Umbral de Confianza

Tipo de búsqueda	min_confidence
Alta seguridad	0.8
Normal	0.6
Exploración	0.0

Proceso de Retrieval

1. Recibir query del usuario/agente

|



2. Clasificar tipo de consulta

|



3. Seleccionar colección y parámetros
|
▼
4. Ejecutar búsqueda en ChromaDB
|
▼
5. Filtrar por confianza mínima
|
▼
6. Ordenar por relevancia
|
▼
7. Formatear resultados con contexto
|
▼
8. Retornar con metadatos útiles

text

Ejemplos de Uso

Decisión Arquitectónica

```
```python
results = memory.retrieve(
 query="microservicios vs monolito para proyecto pequeño",
 collection="all",
 n_results=10,
 min_confidence=0.7
)
```

## Debugging

Python

```
results = memory.retrieve(
 query="connection pool exhausted solución",
 collection="lessons",
 n_results=5,
 min_confidence=0.5
)
```

## Mejores Prácticas

Python

```
results = memory.retrieve(
 query="estructura proyecto Python limpio",
 collection="patterns",
 n_results=7,
 min_confidence=0.8
)
```

## Casos Especiales

### Sin Resultados

Si no hay resultados relevantes:

1. Buscar en colección "all" sin filtro de confianza
2. Expandir a 3 resultados más
3. Si aún nada, no forzar 荐
4. Sugerir guardar nueva lección si el usuario proporciona solución

### Demasiados Resultados

Si hay >20 resultados:

1. Aumentar min\_confidence a 0.8
2. Reducir n\_results a 5
3. Preferir episodios más recientes

### Conflictos

Si hay lecciones contradictorias:

1. Mostrar ambas con su confidence
2. Sugerir al usuario cual seguir basándose en:
  - Mayor confianza
  - Validación más reciente
  - Más aplicaciones exitosas



text

```
`.ai/memory/consolidation.md`

```markdown
# Reglas de Consolidación de Memoria

## Qué es la Consolidación

Proceso de:
1. Unificar memorias similares
2. Reforzar conocimiento validado
3. Descartar información obsoleta o contradictoria
4. Actualizar confianza basándose en evidencia

## Cuándo Consolidar

### Trigger Automático

| Condición | Acción |
|-----|-----|
| STM alcanza 20 items | Consolidar STM → LTM |
| 10+ interacciones sin retrieval | Verificar relevancia |
| Cada 50 interacciones | Auto-consolidación completa |

### Trigger Manual

- Usuario solicita: "consolidar memoria"
- Cambio de proyecto detectado
- Inicio de nueva sesión larga
- 检测到矛盾 (Detección de contradicciones)

## Proceso de Consolidación

### Fase 1: Recolección
```

1. Obtener últimos N episodios (N=100)
 2. Obtener últimas M lecciones (M=200)
 3. Obtener todos los patrones
 4. Revisar STM pendiente

text

```
### Fase 2: Deduplicación
```

Para cada nueva entrada:

1. Buscar entradas similares (distancia < 0.2)
2. Si existe:
 - a. Combinar contenido
 - b. Promediar confianza
 - c. Actualizar timestamp
 - d. Marcar como consolidado
3. Si no existe:
 - a. Agregar como nueva entrada

text

Fase 3: Validación

Para entradas con confianza < 0.5:

1. Marcar como "needs_validation"
2. En próxima sesión:
 - a. Presentar al usuario
 - b. Pedir confirmación o corrección
 - c. Actualizar basándose en feedback

text

Fase 4: Actualización de Confianza

Reglas de actualización:

AFTER successful_application:
confidence = min(1.0, confidence + 0.1)
validation_count += 1

AFTER failed_application:
confidence = max(0.0, confidence - 0.2)
mark_for_review()

AFTER contradictory_evidence:
confidence = confidence * 0.5
create_alert("Posible contradicción detectada")

text

Fase 5: Limpieza

Eliminar entradas con:

- confidence < 0.2 después de 10+ episodios
- No referenciadas en 100+ consolidaciones
- Duplicados exactos (simhash)

text

Consolidación de Episodios

Deduplicación de Decisiones

Si dos episodios tienen:

- Misma decisión (similitud > 0.8)
- Contexto compatible
- Resultados diferentes

→ Crear episodio consolidado con:

- Todos los contextos
- Todos los resultados
- Tags de ambos
- Lecciones combinadas

text

Extracción de Patrones

De cada 10 episodios:

1. Identificar decisiones recurrentes
2. Si misma decisión aplicada 3+ veces exitosamente:
 - a. Crear/actualizar patrón
 - b. Incrementar effectiveness
3. Si decisión aplicada 3+ veces sin éxito:
 - a. Crear/actualizar anti-patrón
 - b. Reducir confidence de lecciones relacionadas

text

Consolidación de Lecciones

Merging de Lecciones Similares

Para lecciones con similitud > 0.7:

1. Combinar contenido (lo más específico)
2. Mantener mayor confianza
3. Combinar categorías
4. Mantener fuentes múltiples
5. Actualizar timestamp

text

Validación Cruzada

Si lección tiene:

- confidence >= 0.8
- validation_count >= 3
- source_count >= 2 proyectos diferentes

→ Marcar como "validated_core_knowledge"
→ Aumentar prioridad en retrieval

text

```
## Métricas de Consolidación

| Métrica | Objetivo |
|-----|-----|
| Deduplicación rate | > 15% |
| Confidence accuracy | > 80% vs feedback |
| Retention rate | > 70% después de 1 mes |
| Retrieval hit rate | > 60% cuando es relevante |
```

Logs de Consolidación

```
Cada consolidación debe registrar:
- Timestamp
- Entradas procesadas
- Duplicados mergeados
- Confidences actualizados
- Entradas eliminadas
- Duración del proceso
```

2024-01-15T10:00:00Z | Consolidation #42

- Episodes: 45 processed, 8 merged, 0 deleted
- Lessons: 120 processed, 15 merged, 2 deleted
- Patterns: 5 processed, 0 merged, 0 deleted
- Duration: 1.2s
- Next scheduled: 2024-01-15T11:00:00Z

.ai/memory/episodic.md

Markdown

Memoria Episódica

Concepto

La memoria episódica registra experiencias específicas del proyecto: decisiones tomadas, contexto, resultados y lecciones derivadas.

Estructura de un Episodio

```
```json
{
 "timestamp": "2024-01-15T10:30:00Z",
 "project": "mi-proyecto",
 "decision": "Usar base de datos PostgreSQL",
 "context": {
 "requirements": "Necesitamos relational data",
 "constraints": ["budget limitado", "equipo pequeño"],
 "alternatives_considered": ["MySQL", "MongoDB"]
 },
 "outcome": {
```

```
{
 "status": "success",
 "metrics": {
 "performance": "improved",
 "maintainability": "high"
 },
 "lessons_learned": [
 "PostgreSQL ofrece mejor soporte JSON para casos híbridos"
],
 "tags": ["database", "architecture", "postgresql"],
 "validado": true
}
```

## Cuándo Crear un Episodio

### Decisiones Arquitectónicas

-  Selección de tecnologías principales
-  Estructura de carpetas
-  Patrones de diseño adoptados
-  Configuraciones críticas

### Decisiones de Código

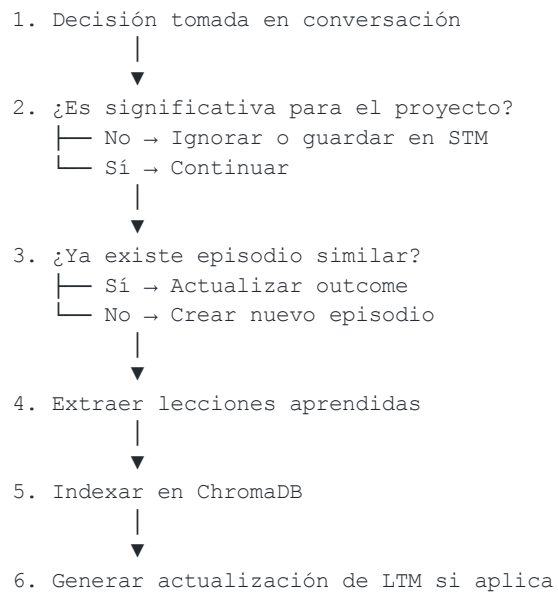
-  Nombres de variables ( STM)
-  Pequeños refactors ( STM)
-  Arquitectura de módulos
-  Patrones de diseño implementados

### Resultados de Experimentos

-  Pruebas A/B de arquitecturas
-  Benchmarking de soluciones
-  Prototipos descartados

## Proceso de Creación

text



## Indexación

Cada episodio se indexa en ChromaDB con:

- Texto completo para búsqueda semántica
- Metadata: project, tags, timestamp, decision\_type
- Link a lecciones derivadas

## Recuperación de Episodios

### Por Proyecto

Python

```
episodes = memory.get_episodes(project="mi-proyecto")
```

## Por Decisión

Python

```
results = memory.retrieve(
 query="elección base de datos",
 collection="episodes",
 n_results=5
)
```

## Por Tags

Python

```
results = memory.retrieve(
 query="database architecture",
 collection="episodes",
 n_results=10
)
```

# Episodios Especiales

## Episodios de Error

JSON

```
{
 "type": "error_resolution",
 "decision": "Migrar de синхронного a async",
 "error_context": "Timeouts en 高峰期",
 "outcome": {
 "status": "success",
 "error_rate_reduction": "80%"
 },
 "lessons_learned": [
 "Siempre diseñar para concurrencia desde el inicio"
]
}
```

## Episodios de Refactor



JSON

```
{
 "type": "refactor",
 "before": "Código monolithico",
 "decision": "Extraer servicios independientes",
 "outcome": {
 "status": "partial",
 "note": "Mejor resultado pero más complejo"
 }
}
```

## Vinculación con LTM

text

```
Episodio → Lecciones aprendidas → LTM (Lessons)
 |
 └→ Patrón detectado → LTM (Patterns)
 |
 └→ Error aprendido → LTM (Anti-patterns)
```

Cada episodio puede generar:

- Lecciones para LTM
- Actualización de patrones existentes
- Nueva evidencia para confianza de lecciones

text

---

```
BLOQUE 5: Tests
```

```
`tests/test_memory.py`
```

```
```python
"""
```

```
Tests para el Sistema de Memoria
```

```
=====
```

```
Valida que el sistema cumple con los requisitos:
```

1. Almacenar episodios con contexto y resultados
2. Recuperar información relevante
3. Consolidar memorias
4. Mejorar con experiencia

```
"""
```

```

import pytest
import os
import shutil
import json
from pathlib import Path
from ai_architect.core.memory import MemorySystem, Episode, Lesson

# ===== Fixtures =====

@pytest.fixture
def memory_system():
    """Crea una instancia limpia del sistema de memoria para cada test"""
    test_dir = "./memory/test_chroma"
    episodic_dir = "./memory/test_episodic"
    semantic_dir = "./memory/test_semantic"

    # Limpiar antes
    for d in [test_dir, episodic_dir, semantic_dir]:
        if os.path.exists(d):
            shutil.rmtree(d)

    mem = MemorySystem(persist_directory=test_dir)
    mem.episodic_path = Path(episodic_dir)
    mem.semantic_path = Path(semantic_dir)
    mem.episodic_path.mkdir(parents=True, exist_ok=True)
    mem.semantic_path.mkdir(parents=True, exist_ok=True)

    yield mem

    # Limpiar después
    for d in [test_dir, episodic_dir, semantic_dir]:
        if os.path.exists(d):
            shutil.rmtree(d, ignore_errors=True)

# ===== Tests de STM =====

class TestShortTermMemory:
    """Tests para memoria de corto plazo"""

    def test_add_to_stm(self, memory_system):
        """Puedo añadir elementos a STM"""
        memory_system.add_to_stm({
            "type": "context",
            "content": "Editando archivo X"
        })
        assert len(memory_system.stm) == 1

    def test_get_stm(self, memory_system):
        """Puedo recuperar elementos de STM"""
        memory_system.add_to_stm({"content": "item1"})
        memory_system.add_to_stm({"content": "item2"})

        stm = memory_system.get_stm()
        assert len(stm) == 2

    def test_get_stm_limit(self, memory_system):
        """Puedo limitar los elementos recuperados"""
        for i in range(10):
            memory_system.add_to_stm({"content": f"item{i}"})

        recent = memory_system.get_stm(n=3)
        assert len(recent) == 3

```

```

def test_stm_max_size_triggers_consolidation(self, memory_system):
    """STM hace auto-consolidación al alcanzar límite"""
    # Añadir más del límite
    for i in range(25):
        memory_system.add_to_stm({
            "content": f"item{i}",
            "learned": f"Aprendí de item{i}"
        })

    # Debería haber hecho consolidación
    assert memory_system.session_interactions == 25

# ===== Tests de Episodios =====

class TestEpisodicMemory:
    """Tests para memoria episódica"""

    def test_store_episode(self, memory_system):
        """Puedo guardar un episodio completo"""
        episode_id = memory_system.store_episode(
            project="test-project",
            decision="Usar PostgreSQL",
            context="Proyecto necesita base de datos relacional",
            outcome="Mejor rendimiento que MySQL",
            tags=["database", "postgresql"],
            lessons_learned=["PostgreSQL tiene mejor soporte JSON"]
        )

        assert episode_id is not None
        assert "test-project" in episode_id

    def test_episode_saved_to_file(self, memory_system):
        """El episodio se guarda en archivo JSON"""
        episode_id = memory_system.store_episode(
            project="test-project",
            decision="Usar PostgreSQL",
            context="Contexto",
            outcome="Resultado"
        )

        files = list(memory_system.episodic_path.glob("*.json"))
        assert len(files) == 1

    def test_episode_indexed_in_chroma(self, memory_system):
        """El episodio se indexa en ChromaDB"""
        memory_system.store_episode(
            project="test-project",
            decision="Usar PostgreSQL",
            context="Contexto",
            outcome="Resultado"
        )

        episodes = memory_system.episodes_collection.get()
        assert len(episodes['ids']) == 1

    def test_retrieve_episodes(self, memory_system):
        """Puedo recuperar episodios por proyecto"""
        # Guardar episodios de diferentes proyectos
        memory_system.store_episode(
            project="proyecto-a",
            decision="Decisión A",
            context="Contexto A",

```

```

        outcome="Resultado A"
    )
    memory_system.store_episode(
        project="proyecto-b",
        decision="Decisión B",
        context="Contexto B",
        outcome="Resultado B"
    )

    episodes = memory_system.get_episodes(project="proyecto-a")
    assert len(episodes) == 1
    assert episodes[0].project == "proyecto-a"

def test_lessons_extracted_from_episode(self, memory_system):
    """Las lecciones del episodio se guardan en LTM"""
    memory_system.store_episode(
        project="test",
        decision="Decisión",
        context="Contexto",
        outcome="Resultado",
        lessons_learned=["Lección importante"]
    )

    lessons = memory_system.lessons_collection.get()
    assert len(lessons['ids']) == 1
    assert "Lección importante" in lessons['documents'][0]

# ===== Tests de LTM =====

class TestLongTermMemory:
    """Tests para memoria de largo plazo"""

    def test_store_lesson(self, memory_system):
        """Puedo guardar una lección"""
        lesson_id = memory_system.store_lesson(
            lesson="Siempre separar concerns",
            category="arquitectura",
            confidence=0.8
        )

        assert lesson_id is not None

    def test_store_pattern(self, memory_system):
        """Puedo guardar un patrón"""
        pattern_id = memory_system.store_pattern(
            pattern="Microservicios con API Gateway",
            context="Proyectos con múltiples servicios",
            effectiveness=0.9
        )

        assert pattern_id is not None

    def test_retrieve_relevant(self, memory_system):
        """Puedo buscar en la memoria y obtener resultados relevantes"""
        # Guardar varios tipos de contenido
        memory_system.store_lesson(
            lesson="PostgreSQL es mejor para datos relacionales",
            category="database"
        )

        memory_system.store_lesson(
            lesson="React es bueno para SPAs",
            category="frontend"
        )

```

```

memory_system.store_pattern(
    pattern="Microservicios",
    context="Proyectos grandes",
    effectiveness=0.8
)

results = memory_system.retrieve(
    query="base de datos relacional",
    n_results=5
)

assert len(results) >= 1
# PostgreSQL debería ser el más relevante
assert any("PostgreSQL" in r['content'] for r in results)

def test_retrieve_with_confidence_filter(self, memory_system):
    """Puedo filtrar por confianza mínima"""
    memory_system.store_lesson(
        lesson="Lección de alta confianza",
        confidence=0.9
    )
    memory_system.store_lesson(
        lesson="Lección de baja confianza",
        confidence=0.3
    )

    results = memory_system.retrieve(
        query="confianza",
        min_confidence=0.8
    )

    assert all(r['metadata'].get('confidence', 0) >= 0.8 for r in results)

# ===== Tests de Retrieval =====

class TestRetrieval:
    """Tests para sistema de recuperación"""

    def test_retrieve_returns_relevance_score(self, memory_system):
        """Los resultados incluyen score de relevancia"""
        memory_system.store_lesson("Python es genial")

        results = memory_system.retrieve(query="lenguaje de programación")

        assert len(results) > 0
        assert 'relevance' in results[0]
        assert 0 <= results[0]['relevance'] <= 1

    def test_retrieve_sorted_by_relevance(self, memory_system):
        """Los resultados están ordenados por relevancia"""
        memory_system.store_lesson("Python para data science")
        memory_system.store_lesson("Java para enterprise")
        memory_system.store_lesson("HTML para web")

        results = memory_system.retrieve(
            query="programación data science",
            n_results=3
        )

        # Python debería ser más relevante que Java
        for i in range(len(results) - 1):
            assert results[i]['relevance'] >= results[i + 1]['relevance']

```

```

def test_retrieve_specific_collection(self, memory_system):
    """Puedo buscar en una colección específica"""
    memory_system.store_lesson("Una lección")
    memory_system.store_pattern("Un patrón")

    results = memory_system.retrieve(
        query="lección",
        collection="lessons"
    )

    assert all(r['collection'] == 'lessons' for r in results)

# ===== Tests de Consolidación =====

class TestConsolidation:
    """Tests para consolidación de memoria"""

    def test_consolidate_runs(self, memory_system):
        """La consolidación se ejecuta sin errores"""
        memory_system.store_lesson("Lección 1")
        memory_system.store_lesson("Lección 2")

        stats = memory_system.consolidate()

        assert stats is not None
        assert 'episodes_processed' in stats

    def test_get_statistics(self, memory_system):
        """Puedo obtener estadísticas del sistema"""
        memory_system.store_episode(
            project="test", decision="D", context="C", outcome="O"
        )
        memory_system.store_lesson("Lección")
        memory_system.store_pattern("Patrón", "Contexto", 0.8)

        stats = memory_system.get_statistics()

        assert stats['episodes_count'] == 1
        assert stats['lessons_count'] == 1
        assert stats['patterns_count'] == 1

# ===== Tests de Integración =====

class TestIntegration:
    """Tests de integración de extremo a extremo"""

    def test_full_memory_cycle(self, memory_system):
        """Puedo hacer el ciclo completo: guardar, recuperar, consolidar"""
        # 1. Guardar episodios de decisiones
        memory_system.store_episode(
            project="proyecto-real",
            decision="Usar FastAPI",
            context="API REST needed",
            outcome="Excelente rendimiento",
            tags=["backend", "api"],
            lessons_learned=["FastAPI es muy rápido"]
        )

        # 2. Guardar conocimiento general
        memory_system.store_lesson(
            lesson="Dependency injection mejora testabilidad",
            category="arquitectura",
            confidence=0.85

```

```

    )

    # 3. Recuperar en contexto similar
    results = memory_system.retrieve(
        query="API REST python rendimiento",
        n_results=5
    )

    # 4. Verificar que encontramos algo relevante
    assert len(results) > 0
    assert any(
        "FastAPI" in r['content'] or "API" in r['content']
        for r in results
    )

    # 5. Consolidar
    stats = memory_system consolidate()
    assert stats['episodes_processed'] > 0

def test_memory_improves_over_time(self, memory_system):
    """El sistema puede 'aprender' de experiencias repetidas"""
    # Primera experiencia
    memory_system.store_lesson(
        lesson="Type hints mejoran código",
        category="python",
        confidence=0.5
    )

    # Segunda experiencia similar (validación)
    memory_system.store_lesson(
        lesson="Type hints hacen código más mantenible",
        category="python",
        confidence=0.6
    )

    # Recuperar debería devolver las lecciones
    results = memory_system.retrieve(
        query="type hints python",
        n_results=2
    )

    assert len(results) >= 1

# ===== Test Runner =====

if __name__ == "__main__":

    pytest.main([__file__, "-v", "--tb=short"])

```

BLOQUE 6: Archivo Principal

main.py

Python

```

#!/usr/bin/env python3
"""
AI Architect - Sistema de Memoria Híbrida
=====
Punto de entrada principal para el sistema de memoria.
"""

import argparse
import json
from ai_architect.core.memory import MemorySystem

def main():
    parser = argparse.ArgumentParser(description="AI Memory System")
    parser.add_argument("--init", action="store_true", help="Inicializar sistema")
    parser.add_argument("--stats", action="store_true", help="Mostrar estadísticas")
    parser.add_argument("--consolidate", action="store_true", help="Ejecutar consolidación")
    parser.add_argument("--query", type=str, help="Buscar en memoria")
    parser.add_argument("--limit", type=int, default=5, help="Límite de resultados")

    args = parser.parse_args()

    mem = MemorySystem()

    if args.init:
        print("✅ Sistema de memoria inicializado")
        print(f"    Directorio: {mem.episodic_path}")
        print(f"    ChromaDB: {mem.client.get().get('name', 'OK')}")

    if args.stats:
        stats = mem.get_statistics()
        print("\n📊 Estadísticas del Sistema")
        print("-" * 40)
        print(f"    Episodios: {stats['episodes_count']}")
        print(f"    Lecciones: {stats['lessons_count']}")
        print(f"    Patrones: {stats['patterns_count']}")
        print(f"    STM actual: {stats['stm_size']}")
        print(f"    Interacciones: {stats['session_interactions']}")

    if args.consolidate:
        print("\n🔄 Ejecutando consolidación...")
        stats = mem.consolidate()
        print(f"    Episodios procesados: {stats['episodes_processed']}")
        print(f"    Lecciones mergeadas: {stats['lessons_merged']}")
        print("✅ Consolidación completada")

    if args.query:
        print(f"\n🔍 Buscando: '{args.query}'")
        results = mem.retrieve(args.query, n_results=args.limit)

        if results:
            for i, r in enumerate(results, 1):
                print(f"\n{i}. [{r['collection']}] (relevancia: {r['relevance']:.2f})")
                print(f"    {r['content'][:200]}...")
        else:
            print("    Sin resultados")

    if not any([args.init, args.stats, args.consolidate, args.query]):
        parser.print_help()

if __name__ == "__main__":
    main()

```

Cómo Usar

1. Instalar dependencias

Bash

```
pip install -r requirements.txt
```

2. Crear la estructura

Bash

```
mkdir -p .ai/memory .ai/reasoning memory/chroma memory/episodic memory/semantic tests
```

3. Guardar los archivos

Copia cada bloque en su archivo correspondiente según la estructura.

4. Ejecutar tests

Bash

```
pytest tests/test_memory.py -v
```

5. Usar desde CLI

Bash

```
# Inicializar
python main.py --init

# Ver estadísticas
python main.py --stats

# Buscar en memoria
python main.py --query "arquitectura python"
```

```
# Consolidar
python main.py --consolidate
```

6. Integrar con OpenCode

Agrega esto en tu `opencode.json`:

JSON

```
{
  "mcpServers": {
    "ai-memory": {
      "command": "python",
      "args": ["ai_architect/core/mcp_server.py"]
    }
  }
}
```

¿Quieres que también te genere el archivo `opencode.json` completo o algún archivo adicional?